

UNIVERSIDAD POLITÉCNICA DE PACHUCA

Ingeniería en Tecnologías de la Información e Innovación Digital

DESARROLLO DE APLICACIONES MÓVILES

Reporte de Práctica

Implementación de Scaffold, TopAppBar y FloatingActionButton en Jetpack Compose

Alumno: Daniel Jimenez Pluma

Grupo: TIID 04 01

Docente: Miguel Angel Montoya Cerro

Lugar: Zempoala, Hidalgo, México

Fecha: 22 de octubre de 2026

Índice general

1 Introducción **5**

1.1 Contextualización 5

1.2 Planteamiento del problema 5

1.3 Objetivo general 5

1.4 Objetivos específicos 5

1.5 Justificación 6

1.6 Estructura del documento 6

2 Marco teórico **6**

2.1 Android Studio y Kotlin 6

2.2 Jetpack Compose y Material Design 3 6

2.3 Scaffold 6

2.4 TopAppBar 7

2.5 FloatingActionButton 7

2.6 Toast y LocalContext 7

2.7 Anotación OptIn 7

2.8 Resumen de los componentes 8

2.9 Antecedentes 8

3 Desarrollo **8**

3.1 Interfaz de partida 8

3.2 Importaciones necesarias 9

3.3 Implementación de Scaffold 11

3.4 Implementación de TopAppBar 11

3.5 Implementación de FloatingActionButton 14

3.6 Mensaje con Toast 16

3.7 Pruebas de la aplicación 16

3.8 Resultado final 17

4 Análisis y argumentación **18**

5 Resultados **19**

6 Explicación del código **19**

6.1 Importaciones nuevas 19

6.2 Anotaciones y contexto 19

6.3 TopAppBar 20

6.4 FloatingActionButton y Toast 20

6.5 Contenido del Scaffold e innerPadding 21

7 Elementos principales utilizados **22**

8 Conclusiones **22**

8.1 Aportes y limitaciones 22

8.2 Recomendaciones y proyecciones 23

9 Trabajo futuro **23**

10 Anexos **23**

10.1 Anexo A. Evidencias adicionales 23

11 Referencias **23**

Índice de ilustraciones

1 Interfaz de partida sin Scaffold. 9
2 Importaciones agregadas para Scaffold, TopAppBar, FloatingActionButton y Toast. 10
3 Función app() con Scaffold y LazyColumn con innerPadding. 11
4 Código del TopAppBar con colores personalizados. 12
5 Vista previa con la barra superior azul. 13
6 Código del FloatingActionButton con icono. 14
7 Vista previa con el botón de acción flotante amarillo. 15
8 Código del Toast dentro del onClick del botón flotante. 16
9 Toast mostrado al presionar el botón de acción flotante. 17
10 Resultado final de la práctica con Scaffold. 18

Índice de tablas

1 Componentes de estructura de pantalla utilizados 8
2 Elementos principales utilizados en la práctica 22

1. Introducción

1.1. Contextualización

El desarrollo de aplicaciones móviles requiere interfaces que, además de mostrar contenido, sigan una estructura reconocible para el usuario. En Android, Jetpack Compose proporciona un enfoque declarativo para construir interfaces mediante funciones escritas en Kotlin (Google, s. f.-a), y Material Design 3 ofrece componentes estándar como barras superiores, botones de acción flotantes y contenedores que organizan la pantalla.

Una de las estructuras más comunes en las aplicaciones Android es una pantalla con una barra superior que muestra el título, un botón de acción flotante para la acción principal y un área central con el contenido. Compose permite construir esta estructura mediante el componente `Scaffold`.

1.2. Planteamiento del problema

En las prácticas anteriores se construyó una interfaz con imagen, textos, una fila horizontal y un contador de likes, pero sin una estructura de pantalla definida: el contenido ocupaba directamente toda el área disponible, sin barra de título ni acciones. Colocar manualmente una barra superior y un botón flotante implicaría calcular posiciones y márgenes, y evitar que el contenido quede oculto debajo de ellos.

La pregunta que guía esta práctica es cómo organizar una pantalla con barra superior, botón de acción flotante y contenido desplazable de forma ordenada, y cómo responder a la acción del usuario con un mensaje. Para resolverla se utilizaron `Scaffold`, `TopAppBar`, `FloatingActionButton` y `Toast`.

1.3. Objetivo general

Implementar una interfaz con Jetpack Compose que utilice `Scaffold` para organizar una barra superior, un botón de acción flotante y un contenido desplazable, comprendiendo el funcionamiento de cada componente.

1.4. Objetivos específicos

- Utilizar `Scaffold` como estructura base de la pantalla.
- Agregar una barra superior con `TopAppBar` y personalizar sus colores.
- Agregar un botón de acción flotante con `FloatingActionButton` y un icono.
- Mostrar un mensaje emergente (`Toast`) al presionar el botón flotante.

- Aplicar el `innerPadding` del `Scaffold` al contenido para evitar que quede oculto.
- Comprender el uso de `LocalContext` y de la anotación `@OptIn`.

1.5. Justificación

La estructura con barra superior, contenido y botón flotante es la base de gran parte de las aplicaciones Android. Comprender cómo se construye con `Scaffold` permite crear pantallas consistentes con Material Design y preparar la interfaz para agregar después otros elementos, como un menú de navegación o una barra inferior.

1.6. Estructura del documento

El documento se organiza de la siguiente manera. Primero se presenta el marco teórico sobre `Scaffold`, `TopAppBar`, `FloatingActionButton` y `Toast`. Después se describe el desarrollo de la práctica con las evidencias obtenidas, seguido del análisis, los resultados y la explicación del código por bloques. Finalmente se incluyen las conclusiones, el trabajo futuro, los anexos y las referencias.

2. Marco teórico

2.1. Android Studio y Kotlin

Android Studio es el entorno de desarrollo utilizado para crear aplicaciones Android; permite editar código, administrar recursos, compilar y probar la aplicación en emuladores o dispositivos físicos. Kotlin es el lenguaje de programación utilizado para desarrollar la aplicación y el que emplea Jetpack Compose para definir interfaces mediante funciones (JetBrains, s. f.).

2.2. Jetpack Compose y Material Design 3

Jetpack Compose es un toolkit moderno para construir interfaces de usuario en Android mediante un enfoque declarativo. La interfaz se define con funciones anotadas con `@Composable`, en lugar de archivos XML (Google, s. f.-a). Los componentes de Material Design 3, como `Scaffold`, `TopAppBar` y `FloatingActionButton`, se encuentran en la librería `material3`.

2.3. Scaffold

`Scaffold` es un componente que proporciona la estructura básica de una pantalla siguiendo Material Design. Cuenta con ranuras (*slots*) para los elementos más comunes, como

la barra superior (`topBar`), la barra inferior (`bottomBar`) y el botón de acción flotante (`floatingActionButton`), además del contenido principal (Google, s. f.-b).

El contenido del `Scaffold` recibe un parámetro llamado `innerPadding`, que indica el espacio ocupado por la barra superior, la barra inferior y las barras del sistema. Este valor debe aplicarse al contenido para que no quede oculto detrás de esos elementos.

2.4. TopAppBar

`TopAppBar` es la barra superior de la aplicación. Muestra un título y puede incluir iconos de navegación y de acciones (Google, s. f.-c). Sus colores se personalizan mediante `TopAppBarDefaults.topAppBarColors`, con parámetros como `containerColor` para el fondo y `titleContentColor` para el color del título.

2.5. FloatingActionButton

`FloatingActionButton` es un botón circular que flota sobre el contenido y representa la acción principal de la pantalla (Google, s. f.-d). Recibe una función `onClick` que se ejecuta al presionarlo, y su aspecto se configura con `containerColor` y `contentColor`. En su interior se coloca habitualmente un `Icon`.

2.6. Toast y LocalContext

`Toast` es un mensaje breve y temporal que se muestra sobre la interfaz sin interrumpir al usuario (Google, s. f.-e). Para crearlo se necesita un `Context`, que en Compose se obtiene con `LocalContext.current`.

2.7. Anotación OptIn

Algunos componentes de la librería se marcan como experimentales, lo que significa que su diseño puede cambiar en versiones futuras. Para utilizarlos, la función debe declarar explícitamente que acepta ese riesgo mediante `@OptIn(ExperimentalMaterial3Api::class)`.

2.8. Resumen de los componentes

Tabla 1: Componentes de estructura de pantalla utilizados

Componente	Función
`Scaffold`	Estructura base de la pantalla con ranuras para barras y botón flotante.
`TopAppBar`	Barra superior con el título de la pantalla.
`FloatingActionButton`	Botón circular flotante para la acción principal.
`Toast`	Mensaje breve y temporal sobre la interfaz.
`LocalContext`	Proporciona el contexto de Android dentro de un composable.

2.9. Antecedentes

Esta práctica da continuidad a las prácticas anteriores sobre `Column`, `LazyColumn`, `LazyRow` y estados, en las que se construyó la interfaz con imagen, textos, una fila horizontal y un contador de likes. En esta práctica se conserva ese contenido y se coloca dentro de una estructura de pantalla con `Scaffold`. Como material de apoyo se utilizó un video tutorial de Jetpack Compose (AristiDevs, [año]).

3. Desarrollo

3.1. Interfaz de partida

Para esta práctica se reutilizó el proyecto `ColumnAplication`, que contiene una `LazyColumn` con fondo rojo, una imagen, textos, un contador de likes y una fila horizontal. Esta interfaz no cuenta con barra superior ni botones de acción.

```
@Composable
fun Likes() {
    var likes by rememberSaveable { mutableStateOf(0)}

    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
        horizontalArrangement = Arrangement.Center,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(
            imageVector = Icons.Filled.Favorite,
            contentDescription = "Me gusta",
            tint = Color.White,
            modifier = Modifier
                .size(48.dp)
                .clickable { likes++ }
        )
        Text(
            text = "$likes",
            fontSize = 24.sp,
            color = Color.White,
            modifier = Modifier.padding(start = 8.dp)
        )
    }
}

@Preview(showBackground = true)
@Composable
fun app() {
    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .background(Color.Red)
    ) {
        item {
            Image(
                modifier = Modifier
                    .fillMaxWidth()
                    .height(400.dp),
                painter = painterResource(id = R.drawable.potro),
                contentDescription = "Logo"
            )
```

Ilustración 1: Interfaz de partida sin Scaffold.

3.2. Importaciones necesarias

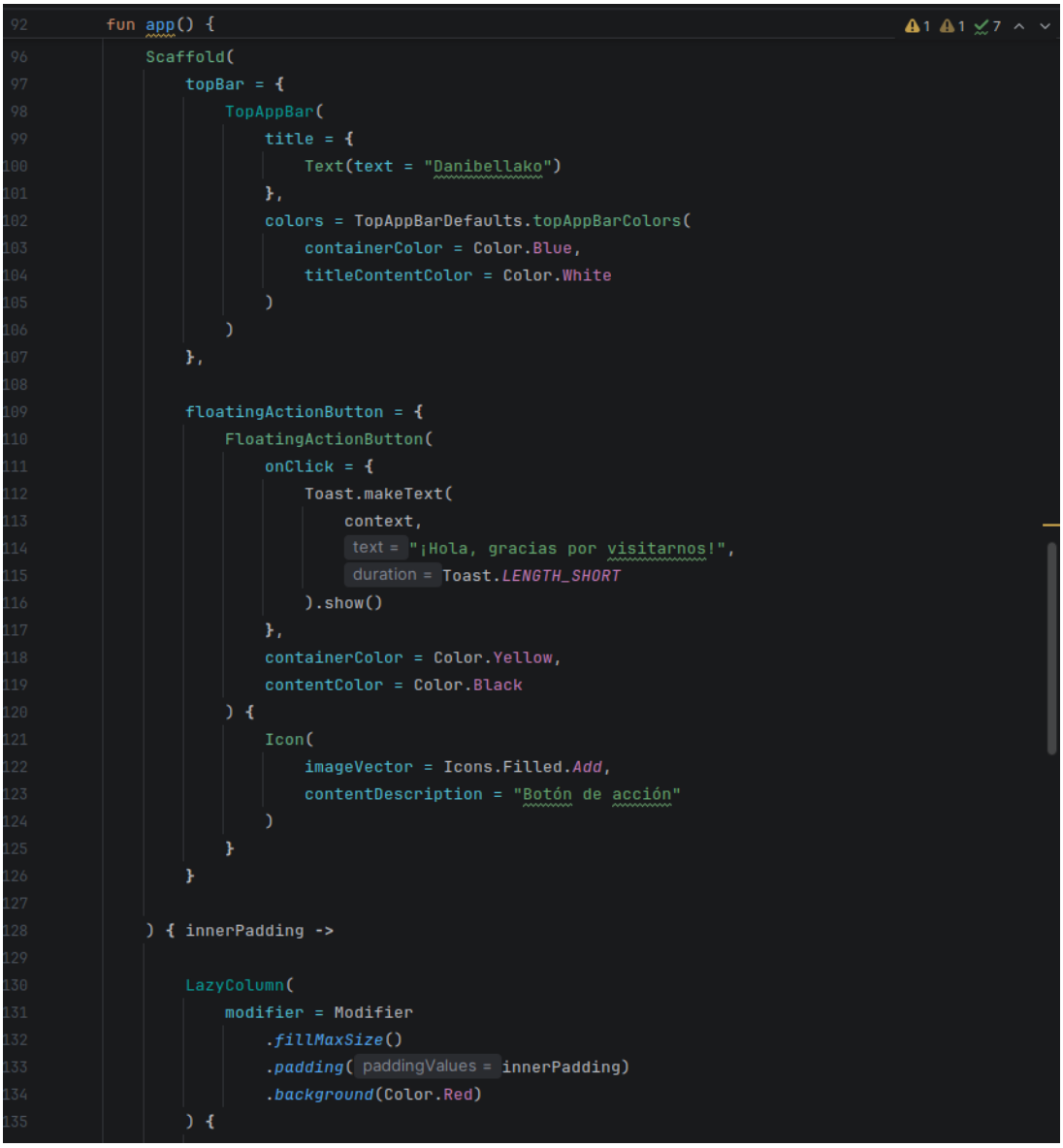
Se agregaron las importaciones para los nuevos componentes: `Scaffold`, `TopAppBar`, `TopAppBarDefaults`, `FloatingActionButton`, `ExperimentalMaterial3Api`, `Icons.Filled.Add`, `LocalContext` y `Toast`.

```
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.Favorite
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FloatingActionButton
import androidx.compose.material3.Icon
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.columnaplication.ui.theme.ColumnAplicationTheme
```

Ilustración 2: Importaciones agregadas para Scaffold, TopAppBar, FloatingActionButton y Toast.

3.3. Implementación de Scaffold

La `LazyColumn` de la función `app()` se colocó dentro de un `Scaffold`, que pasó a ser la estructura base de la pantalla. El contenido del `Scaffold` recibe el parámetro `innerPadding`, que se aplicó a la `LazyColumn` mediante `Modifier.padding(innerPadding)` para que el contenido no quede oculto debajo de la barra superior. Además, se agregó la anotación `@OptIn(ExperimentalMaterial3Api::class)`.



```
fun app() {
    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                    Text(text = "Danibellako")
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = Color.Blue,
                    titleContentColor = Color.White
                )
            )
        },

        floatingActionButton = {
            FloatingActionButton(
                onClick = {
                    Toast.makeText(
                        context,
                        text = "¡Hola, gracias por visitarnos!",
                        duration = Toast.LENGTH_SHORT
                    ).show()
                },
                containerColor = Color.Yellow,
                contentColor = Color.Black
            ) {
                Icon(
                    imageVector = Icons.Filled.Add,
                    contentDescription = "Botón de acción"
                )
            }
        }

    ) { innerPadding ->

        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .padding( paddingValues = innerPadding)
                .background(Color.Red)
        ) {
```

Ilustración 3: Función app() con Scaffold y LazyColumn con innerPadding.

3.4. Implementación de TopAppBar

En la ranura `topBar` del `Scaffold` se agregó un `TopAppBar` con el título "Danibellako". Sus colores se personalizaron con `TopAppBarDefaults.topAppBarColors`: fondo azul (`containerColor = Color.Blue`) y título en color blanco (`titleContentColor =`

`Color.White`).

```
topBar = {
    TopAppBar(
        title = {
            Text(text = "Danibellako")
        },
        colors = TopAppBarDefaults.topAppBarColors(
            containerColor = Color.Blue,
            titleContentColor = Color.White
        )
    )
},
```

Ilustración 4: Código del TopAppBar con colores personalizados.

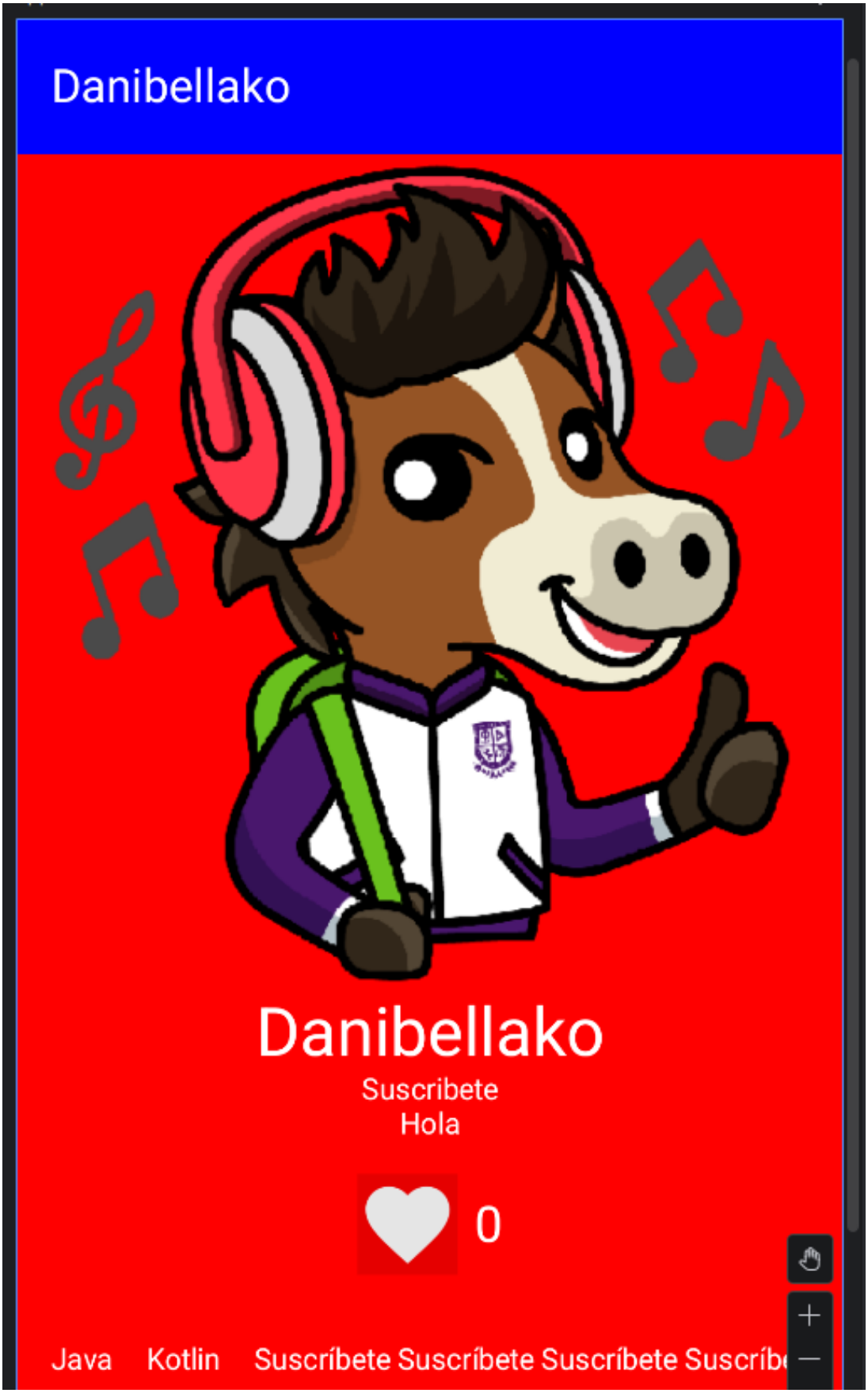


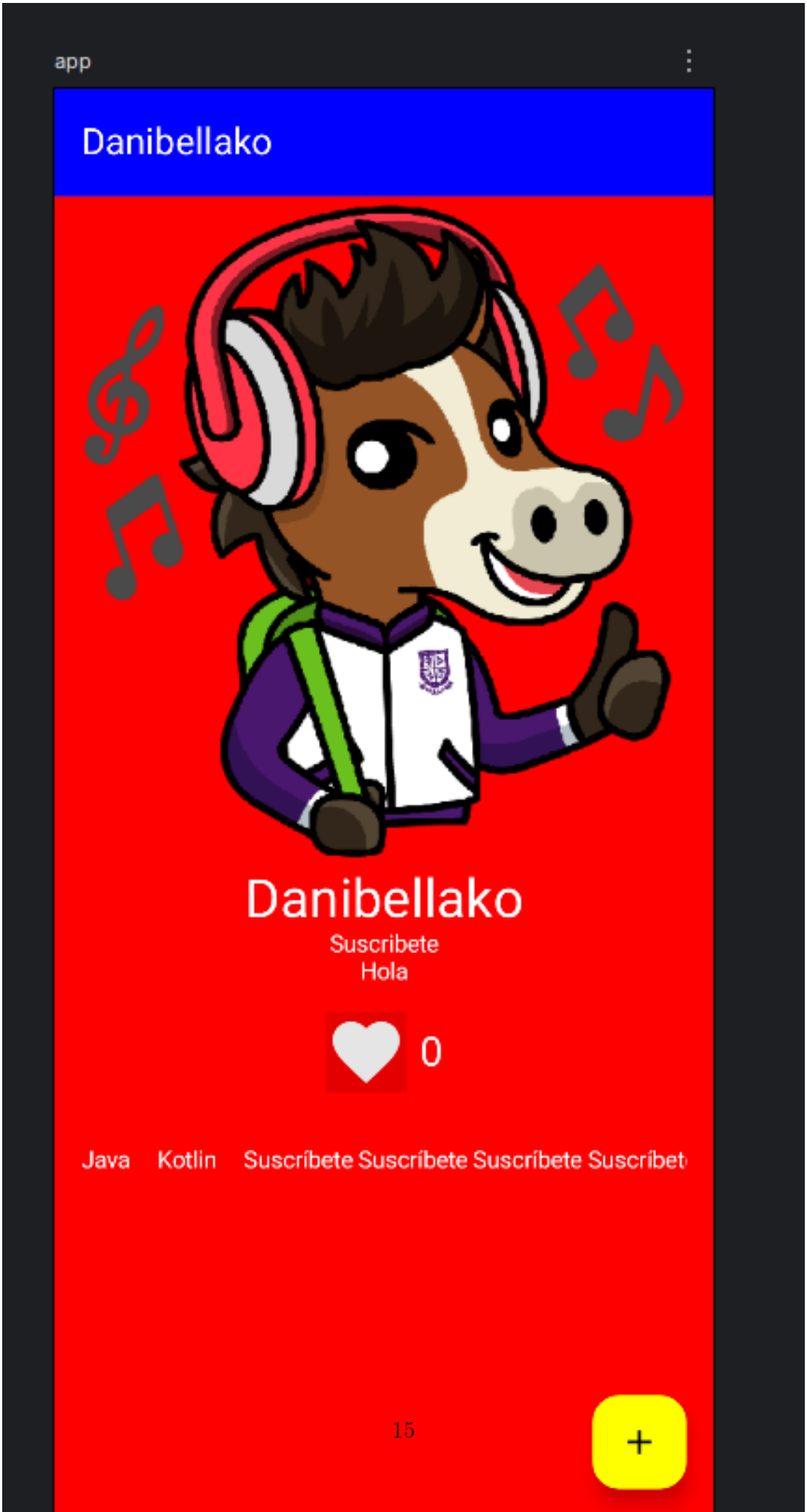
Ilustración 5: Vista previa con la barra superior azul.

3.5. Implementación de FloatingActionButton

En la ranura `floatingActionButton` se agregó un `FloatingActionButton` con fondo amarillo (`containerColor = Color.Yellow`), contenido negro (`contentColor = Color.Black`) y un icono de suma (`Icons.Filled.Add`).

```
floatingActionButton = {
    FloatingActionButton(
        onClick = {
            Toast.makeText(
                context,
                text = "¡Hola, gracias por visitarnos!",
                duration = Toast.LENGTH_SHORT
            ).show()
        },
        containerColor = Color.Yellow,
        contentColor = Color.Black
    ) {
        Icon(
            imageVector = Icons.Filled.Add,
            contentDescription = "Botón de acción"
        )
```

Ilustración 6: Código del FloatingActionButton con icono.


app
Danibellako
Danibellako
Suscribete
Hola
0
Java
Kotlin
Suscríbete Suscríbete Suscríbete Suscríbet
+

3.6. Mensaje con Toast

Para que el botón respondiera al usuario, se obtuvo el contexto con `val context = LocalContext.current` y se utilizó en el parámetro `onClick` para mostrar un `Toast` con el mensaje "¡Hola, gracias por visitarnos!" y duración corta (`Toast.LENGTH_SHORT`).

```
onClick = {
    Toast.makeText(
        context,
        text = "¡Hola, gracias por visitarnos!",
        duration = Toast.LENGTH_SHORT
    ).show()
},
containerColor = Color.Yellow,
contentColor = Color.Black
```

Ilustración 8: Código del Toast dentro del onClick del botón flotante.

3.7. Pruebas de la aplicación

Se ejecutó la aplicación en el emulador y se presionó el botón flotante para comprobar que el mensaje aparece sobre la interfaz.

```
1 Usage
@Composable
fun Likes() {
    var likes by rememberSaveable { mutableStateOf( value = 0)}

    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding( all = 16.dp),
        horizontalArrangement = Arrangement.Center,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(
            imageVector = Icons.Filled.Favorite,
            contentDescription = "Me gusta",
            tint = Color.White,
            modifier = Modifier
                .size(48.dp)
                .clickable { likes++ }
        )
        Text(
            text = "$likes",
            fontSize = 24.sp,
            color = Color.White,
            modifier = Modifier.padding(start = 8.dp)
        )
    }
}
```

Ilustración 9: Toast mostrado al presionar el botón de acción flotante.

3.8. Resultado final

La interfaz final muestra la barra superior azul con el título, el contenido desplazable con la imagen, los textos, el contador de likes y la fila horizontal, y el botón de acción flotante amarillo en la parte inferior derecha.

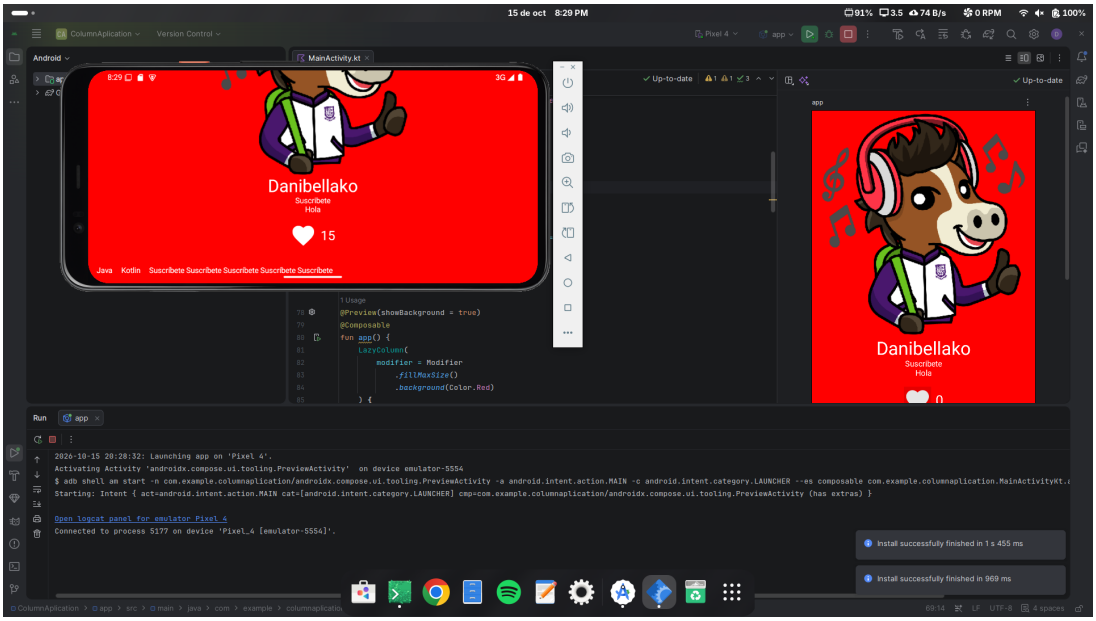


Ilustración 10: Resultado final de la práctica con Scaffold.

4. Análisis y argumentación

El desarrollo de la práctica permitió observar cómo `Scaffold` organiza la pantalla en zonas definidas. Cada elemento se coloca en su ranura (`topBar`, `floatingActionButton` y contenido) y el componente se encarga de ubicarlo correctamente, sin necesidad de calcular posiciones manualmente (Google, s. f.-b).

Un punto importante es el uso de `innerPadding`. Como la barra superior ocupa espacio en la parte alta de la pantalla, el contenido debe recibir ese valor para empezar debajo de ella. Por ello se aplicó `Modifier.padding(innerPadding)` a la `LazyColumn`. El orden de los modificadores también importa: al colocar `padding(innerPadding)` antes de `background(Color.Red)`, el fondo rojo solo cubre el área que corresponde al contenido y no se extiende debajo de la barra superior.

Respecto a `TopAppBar`, la personalización de colores mediante `TopAppBarDefaults.topAppBarCo` permitió modificar el fondo y el título sin crear un componente propio. Este componente requiere la anotación `@OptIn(ExperimentalMaterial3Api::class)`, ya que forma parte de las APIs experimentales de Material 3.

En el botón flotante, la función `onClick` permitió ejecutar una acción al presionarlo. Para mostrar el `Toast` fue necesario obtener un contexto con `LocalContext.current`, ya que este mensaje pertenece al sistema de Android y requiere un `Context` para crearse (Google, s. f.-e).

Por último, la combinación de estos componentes con el contenido de las prácticas anteriores, incluido el contador con `rememberSaveable`, demostró que el contenido

existente puede integrarse a una nueva estructura sin necesidad de reescribirlo.

5. Resultados

Como resultado de la práctica se obtuvo una aplicación Android desarrollada con Kotlin y Jetpack Compose con una estructura de pantalla completa: una barra superior azul con el título, un área de contenido desplazable y un botón de acción flotante amarillo.

Al presionar el botón flotante se muestra un mensaje emergente con el texto "¡Hola, gracias por visitarnos!". El contenido de las prácticas anteriores se conserva y queda ubicado debajo de la barra superior gracias al `innerPadding`. Se cumplió el objetivo de comprender cómo se organiza una pantalla con `Scaffold`.

6. Explicación del código

A continuación se explican por bloques las partes del código desarrollado en esta práctica. Las funciones `MainActivity` y `Likes()` no cambiaron respecto a la práctica anterior.

6.1. Importaciones nuevas

```
1                import android.widget.Toast
2                import androidx.compose.material.icons.filled.Add
3                import androidx.compose.material3.ExperimentalMaterial3Api
4                import androidx.compose.material3.FloatingActionButton
5                import androidx.compose.material3.Scaffold
6                import androidx.compose.material3.TopAppBar
7                import androidx.compose.material3.TopAppBarDefaults
8                import androidx.compose.ui.platform.LocalContext
9
```

Descripción. Son las importaciones agregadas respecto a la práctica anterior. `Scaffold`, `TopAppBar`, `TopAppBarDefaults` y `FloatingActionButton` son los componentes de Material 3; `ExperimentalMaterial3Api` permite declarar el uso de APIs experimentales; `Icons.Filled.Add` es el icono de suma del botón; `LocalContext` da acceso al contexto de Android; y `Toast` muestra el mensaje emergente.

6.2. Anotaciones y contexto

```
1                @OptIn(ExperimentalMaterial3Api::class)
2                @Preview(showBackground = true)
3                @Composable
```

```
4               fun app() {
5
6                       val context = LocalContext.current
7
8                       // Scaffold ...
9               }
10
```

Descripción. `@OptIn(ExperimentalMaterial3Api::class)` indica que la función usa componentes experimentales de Material 3, necesario para `TopAppBar`. `@Preview` permite ver la interfaz en Android Studio y `@Composable` la identifica como parte de la interfaz. La línea `val context = LocalContext.current` obtiene el contexto de Android, que se usará más adelante para crear el `Toast`.

6.3. TopAppBar

```
1               topBar = {
2                       TopAppBar(
3                       title = {
4                               Text(text = "Danibellako")
5                       },
6                       colors = TopAppBarDefaults.topAppBarColors(
7                       containerColor = Color.Blue,
8                       titleContentColor = Color.White
9                       )
10                      )
11              },
12
```

Descripción. El parámetro `topBar` del `Scaffold` recibe la barra superior. `title` define el contenido del título, en este caso un `Text` con "Danibellako". El parámetro `colors` personaliza la barra con `TopAppBarDefaults.topAppBarColors`: `containerColor` establece el fondo azul y `titleContentColor` el color blanco del título.

6.4. FloatingActionButton y Toast

```
1               floatingActionButton = {
2                       FloatingActionButton(
3                       onClick = {
4                               Toast.makeText(
5                               context,
```

```
                        "¡Hola, gracias por visitarnos!",
                        Toast.LENGTH_SHORT
                        ).show()
                },
                containerColor = Color.Yellow,
                contentColor = Color.Black
                ) {
                        Icon(
                        imageVector = Icons.Filled.Add,
                        contentDescription = "Botón de acción"
                        )
                }
        }

```

Descripción. El parámetro `floatingActionButton` del `Scaffold` recibe el botón flotante. Su parámetro `onClick` se ejecuta al presionarlo: `Toast.makeText` crea el mensaje con el contexto, el texto y la duración corta (`Toast.LENGTH_SHORT`), y `.show()` lo muestra. `containerColor` define el fondo amarillo y `contentColor` el color negro del icono. Dentro del botón se coloca un `Icon` con `Icons.Filled.Add`, y `contentDescription` aporta accesibilidad.

6.5. Contenido del Scaffold e innerPadding

```
        ) { innerPadding ->

                LazyColumn(
                modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .background(Color.Red)
                ) {
                        // imagen, textos, Likes() y LazyRow
                }
        }

```

Descripción. La llave final del `Scaffold` recibe el contenido principal, que a su vez recibe el parámetro `innerPadding`. La `LazyColumn` lo aplica con `padding(innerPadding)`, de modo que su contenido empieza debajo de la barra superior. El orden de los modificadores es importante: primero `fillMaxSize()`, luego el

`padding` y al final el `background`, por lo que el fondo rojo solo cubre el área de contenido. Dentro de la `LazyColumn` se conservan la imagen, los textos, el contador `Likes()` y la fila horizontal.

7. Elementos principales utilizados

Tabla 2: Elementos principales utilizados en la práctica

Elemento	Función
`Scaffold`	Estructura base de la pantalla con ranuras para barra superior y botón flotante.
`topBar`	Ranura del Scaffold para la barra superior.
`TopAppBar`	Barra superior con el título de la pantalla.
`TopAppBarDefaults.topAppBarColors`	Personalizan los colores de la barra superior.
`floatingActionButton`	Ranura del Scaffold para el botón de acción flotante.
`FloatingActionButton`	Botón circular flotante para la acción principal.
`onClick`	Función que se ejecuta al presionar el botón.
`Icon` e `Icons.Filled.Add`	Muestran el icono de suma dentro del botón.
`Toast`	Muestra un mensaje breve y temporal.
`LocalContext`	Proporciona el contexto de Android dentro de un composable.
`innerPadding`	Espacio que ocupan las barras, que se aplica al contenido.
`@OptIn`	Acepta el uso de APIs experimentales.
`LazyColumn`	Contenido principal con desplazamiento vertical.

8. Conclusiones

8.1. Aportes y limitaciones

La realización de esta práctica permitió comprender cómo se organiza una pantalla completa en Jetpack Compose mediante `Scaffold`, con una barra superior personalizada, un botón de acción flotante y un contenido desplazable. También se comprendió la importancia de aplicar el `innerPadding`, el uso de `LocalContext` para mostrar un `Toast` y la necesidad de `@OptIn` para APIs experimentales.

Entre las limitaciones se encuentra que los colores se asignaron de forma fija y no mediante el tema de la aplicación, que el texto del título y del mensaje está escrito directamente en el código y que el botón solo muestra un mensaje sin ejecutar una acción real. Además, `Toast` pertenece al sistema de vistas tradicional de Android y no es un componente propio de Compose.

8.2. Recomendaciones y proyecciones

Se recomienda utilizar los colores del tema de Material (`MaterialTheme.colorScheme`) en lugar de colores fijos, mover los textos a recursos de cadenas y evaluar el uso de `Snackbar` con `SnackbarHost`, que es el componente de mensajes recomendado en Compose.

9. Trabajo futuro

Como continuación de esta práctica se pueden implementar las siguientes mejoras:

- Agregar iconos de navegación y de acciones a la barra superior.
- Incorporar una barra de navegación inferior mediante la ranura `bottomBar`.
- Sustituir el `Toast` por un `Snackbar`.
- Hacer que el botón flotante agregue elementos a la lista.
- Utilizar los colores del tema de la aplicación.

10. Anexos

10.1. Anexo A. Evidencias adicionales

Las diez ilustraciones incluidas en el desarrollo del documento representan las principales etapas realizadas durante la práctica. Las imágenes utilizadas corresponden a los archivos `1.png` a `10.png`.

11. Referencias

- AristiDevs. ([año]). *[Scaffold (TOOLBAR y FAB) en JETPACK COMPOSE - Tutorial Android JETPACK COMPOSE desde cero [2021]]* [Video]. YouTube. [https://youtu.be/jvfZWtoggwc?si=zX7wyXoSiE9EKKvU]
- Google. (s. f.-a). *Jetpack Compose*. Android Developers. https://developer.android.com/compose

- Google. (s. f.-b). *Scaffold*. Android Developers. https://developer.android.com/develop/ui/compose/components/scaffold
- Google. (s. f.-c). *Barras de aplicaciones*. Android Developers. https://developer.android.com/develop/ui/compose/components/app-bars
- Google. (s. f.-d). *Botón de acción flotante*. Android Developers. https://developer.android.com/develop/ui/compose/components/fab
- Google. (s. f.-e). *Descripción general de los avisos (Toast)*. Android Developers. https://developer.android.com/guide/topics/ui/notifiers/toasts
- JetBrains. (s. f.). *Kotlin documentation*. Kotlin. https://kotlinlang.org/docs/home.html
- Jiménez Pluma, D. (2026). *Desarrollo de Aplicaciones Móviles* [Repositorio de código]. GitHub. https://github.com/danieljiplu-007/Desarrollo-de-Aplicaciones-Movi
- Universidad Politécnica de Pachuca. (s. f.). *Guía de estructura y desarrollo de documentos académicos*. Material proporcionado para la asignatura.